

Distributed Load Balancing

We aim to build a distributed, fault-tolerant L7 load balancing system where multiple backend servers serve requests and the system must remain available despite failures. It must handle environments where backend instances can dynamically scale up or down, failures are common, and traffic patterns can vary.

System Overview

At a high level, the architecture follows:

Clients → DNS Server → Load Balancers → Backend Servers

- **Clients** query DNS and send requests
- **DNS server** returns multiple load balancer IPs
- **Load balancers**, consisting of 2 parts
 - Data Forwarders
 - Raft based Controllers
- **Backend servers** handle application logic

Each load balancer is an L7 proxy that receives client requests and forwards them to backend servers. We use **Raft** to ensure every load balancer maintains a globally consistent view of backend health and system config.

For the sake of efficiency, we avoid the naive approach where every load balancer probes every backend. We instead use a **subset assignment strategy** based on consistent hashing. Each backend is assigned to a small number of load balancers, and only those load balancers are responsible for probing it. The results are then shared across the cluster using Raft. The system also supports dynamic configuration changes, such as switching the load balancing algorithm.

Data

Server registry

It is a map of service names to a list of backend instances that provide that service. Each instance includes IP, port and TTL, and is written on register or heartbeat messages from the backend servers. This is used by LBs to discover which backends exist.

Health Observations

Generated by all load balancers by probing the subset of backends assigned to it. These observations include success rate, latency, and error rate. This information is then **replicated across all LBs using Raft**.

Aggregated Health State

From these observations, each load balancer computes an aggregated health state for every backend. This is what is used for routing decisions.

Global Configuration

This includes parameters such as the load balancing algorithm, health thresholds, and probing intervals. **This configuration is replicated via Raft**, ensuring that all load balancers operate under the same settings.

Load Balancer Cluster

Finally, the system maintains the **set of active load balancers**, which is managed through the Raft protocol.

Types of Users

1. **Service owners** are responsible for managing backend servers (register and deregister servers)
2. **System Administrators** manage the load balancing system itself. They can monitor system health, simulate failures, and update configuration parameters.
3. **Clients** are external users who interact with the system by sending HTTP requests.

Internal Architecture

The high-level architecture is simply:

Clients → DNS Server → Load Balancers → Backend Servers

DNS Server

The DNS server maps a service name to multiple load balancer addresses. This allows clients to distribute requests and retry if a load balancer is unavailable.

Load Balancer Design

Each Load Balancer has two roles:

- **Forwarder (Data Plane):** receives HTTP requests and routes them to backend servers
- **Controller (Control Plane):** handles service discovery, health checking, and configuration

All load balancers form a **Raft cluster**, which provides both consistency and fault tolerance. One node acts as the leader, and the others act as followers. When a load balancer performs a health check on a backend, it sends the result to the leader, which appends it to the Raft log. Once the update is replicated to a majority of nodes, it is committed and applied by all load balancers in the same order.

Service Discovery

The service registry keeps track of active backend instances. Backends register themselves and periodically refresh their presence, while load balancers use this information to discover where to route requests. The registry is replicated across all load balancers via Raft. (Similarly global configuration is shared, and because these updates are very infrequent, they don't add much to the network traffic)

Health Checking

Health checking is distributed across load balancers.

Each backend is assigned to a small number of load balancers using **consistent hashing**. Only these assigned load balancers actively probe it. This reduces redundant checks while still providing redundancy.

Each load balancer maintains a local health view based on:

- active probes (HTTP checks)
- passive signals (request failures/successes)

This is shared across the system using Raft.

Request Routing

When a request arrives:

1. Filter out unhealthy backends
2. Select a backend using a load balancing algorithm
3. Forward the request

Routing decisions are made locally at each load balancer using one of these algorithms-

1. Round robin
2. Least request
3. Consistent Hashing (Maglev)

Fault Tolerance

Load Balancer

If a load balancer goes down, it's detected through missed Raft heartbeats. A new leader is elected automatically if needed, and the failed node simply drops out of the system. Since load balancers compute their responsibilities using consistent hashing, the work that node was doing (like health checking certain backends) is picked up by others without any explicit coordination.

On the client side, a request to a failed load balancer may fail, but clients receive multiple LB addresses from DNS, so they can retry with another one.

Backend Servers

Backend failures are first detected locally by the load balancers responsible for probing them. This information becomes available to the entire system eventually, and unhealthy backend servers are avoided globally.